Contents

Introduction **2**
Foundation of Coding Interviews . . . 2
How to Use LeetCode . . . 3

Bit Manipulation **4**

Math **5**

List **6**

String **8**

Set **9**

Dictionary **10**

Linked List **11**
Tips . . . 12

Deque / Stack / Queue **12**

Heap **14**

Divide and Conquer **15**

Sort **15**

Binary Search **18**

Self-Balancing Binary Search Tree **19**

Binary Tree **20**
Traversal . . . 20

Graph **23**
Categories & Must-Knows . . . 24
Representation . . . 26
Search . . . 27
Advanced Graph Topics . . . 28

Backtracking **29**
Difference between Recurrence Relation . . . 29
Tips . . . 29

Dynamic Programming **29**
Tips . . . 32

Greedy **32**
Tips . . . 33

Two Pointers **33**

Complexity Theory **33**

Concurrency **33**

Parallelism **34**
Database Isolation Levels 35

Remarks on Concurrency and Parallelism **36**
Coding Tips 37

Introduction

Foundation of Coding Interviews

Never code first:

1. Clarify the problem with given examples to make sure you are on the right page with the interviewer. If it's a design question (eg. design LRU cache) make sure to clarify interfaces of the class you are implementing along with the trade-offs of possible approaches.
2. Discuss edge (*Base Case* in *Recursion*) / extreme cases. The cases defined here along with the given example cases become the test cases in the following steps. Define the input / output of the cases found.
3. When you came up with an algorithm, walk through the steps with drawings by using the test cases you've come up with in previous steps. Write down key points in each step with pseudocode if you need to.
4. Discuss time / space complexity.
5. Implement code.
6. Test with example and dry-run with edge / extreme cases.
7. Optimize and reiterate.

Start drawing things no matter what:

An average person cannot fit an entire solution process of a complex problem in their brain, since it is hard to keep track of states while coming up with logic for solution (multitasking is not a nature for everyone). For this reason, start out by drawing anything on whiteboard (even by reciting the question). This will help your brain to focus on the solution logic.

Become a debugger yourself when testing:

When debugging with debugging tools, we have an observer window to observe variables or stack values for each step. You need to be the debugger yourself to show how the input is processed in each step.

Think of a suboptimal solution first:

1. Always come up with a suboptimal (can be even brute force) solution first, since it has the advantage of
   - You're guaranteed minimum points, as it can be a checkpoint (better than not being able to implement an optimal approach on first try).

    - The interviewer can offer hints from this checkpoint if you cannot think of further optimal approach.
    - If the suboptimal approach gets improved, there will be extra points for the improvement process itself rather than jumping to the most optimal solution.
    - Some problems (like Dynamic Programming problems) are mostly half-solved, when you have a working suboptimal solution because the optimal approach is to just revise the code a bit.
2. Explain the steps in pseudocode but do not implement it yet. If the suboptimal approach is either too simple or complex to code, ask the interviewer if we can jump to better solution. This is better than not communicating the suboptimal approach at all and later mess up with the optimal approach.
3. If you don't have much time left, but you know an optimisation technique, quickly just mentioning the approach along with time / space complexity can get you some extra points.

Stick to a coding style:

- Coding style seems trivial, but it may give the interviewer the impression of how much you care about code quality.
- Google has great coding standards you can use as a base.

Smartly name parameter, variables and methods:

- This also gives the interviewer the impression of how much you care about code quality.

Know your language libraries:

- You need to be master of the language you've chosen for the interview. You will not be able to look up the internet for core APIs, therefore be sure to learn them by heart.
- Showing good knowledge of the language of your choice conveys good impression to the interviewer.

Make use of non-implemented methods with robust interface when sketching higher level implementation:

- This can greatly help you focus on sketching the higher level code. You can tell your interviewer you will implement the empty methods later.
- You can demonstrate simple TDD lifecycle since empty methods will fail the test, and you can make it pass by implementing it.
- The interviewer may give you extra points for testability, modularization and readability of code.

Think of the interviewer as your colleague:

- Consider the interview as a laid back discussion in a meeting.
- Don't choke and always communicate with interviewer **out loud** even if you don't know where you're going.

How to Use LeetCode

- Difficulties (easy, medium, hard) can be an overall indicator, yet do not get too much attached to it since difficulty is subjective. Easy labeled problems can turn into medium if further optimisation is required, or mediums can turn easy if the problem is well-known. Interviewer's expectation will also vary based on the these elements.

- Learn various ways to solve a particular problem. Only aiming for the most optimal solution will prevent you from learning many other creative ways to solve a problem. Learning various ways to solve a particular problem will definitely strengthen your fundamentals and later help you approach problems you have never seen before. For example, if you are introduced with a solution of a Dynamic Programming problem with Tabulation, try to solve it with Memoization. Remember, **Quality over Quantity**.
- There are questions that are a tweak of another problem and look something like 'Some Problem I' and 'Some Problem II'. Solve the original version 'Some Problem I' and others in ordering of number, as this will give you the feeling of solving a problem and it's follow-ups.
- Do not hold on to one problem for more than 1 hour. If the bar is too high, break open a hint or go to solutions.
  - Not looking at the solution is a good practice but there is also limited time for preparation. Also, some problems simply require specific knowledge to nail it, therefore as a rule of thumb
    * Look at the solution although never skip implementing the solution yourself. Oftentimes there are specific knowledge or tricks that you will learn from a problem.
    * Understand every aspect of the solution (Why was recursion used here? /Why is parameter passed this way? etc.).
    * Come back to the problem 1 week later and solve from scratch. If you can, you have completely understood the solution.
  - Exceptionally hard problems (acceptance rate lower than 20%) are mostly not expected to be solved optimally within time, so try to focus on pattern recognition and a working solution.
  - When you're really short of time, follow the foundation above but leave out the coding part. Compare your solutions with the LeetCode solutions, take notes and move on.
- Code on *Google Docs* or *Coder Pad* since it does not rely on LeetCode's helper functionalities. Coding on these plain text editors will be the more close to the actual interviewing environment and help you develop the habit of
  - Defining the method signature yourself (you can't rely on ready-made method signature like in LeetCode in interviews).
  - Coming up with generic and edge / extreme case inputs yourself (you can't rely on ready-made test cases like in LeetCode in interviews).
  - Testing your application line-by-line with the input you came up with (You can't compile / run / debug your code like in LeetCode in interviews). Especially in LeetCode, there is a 'run' button which is very easy to abuse leading to premature commitment of your code. This is a huge **Red Flag**. Only once you're done coding in the plain text editor, copy-paste and run the code on LeetCode to confirm any missed test cases.
- Finally, NeetCode 150 covers almost all patterns of questions. Digest every pattern and approaches in this list and implement it yourself. If you can comfortably solve at least medium questions in 20 minutes, you are ready-for-interview.

Bit Manipulation

Before diving in, please review *Two's Complement* which is used for representing signed integers. Python has many built-in types, and they are all objects (there are no such thing as primitive types like Java or C++). However, primary ones that are most used are

- `bool`: Either `True` or `False`.

- `int`: Has infinite precision, meaning it can hold very large number that is bound to system memory (`2^63-1` for 64-bit system).
- `float`: Does not have infinite precision (follows *IEEE 754 Floating-Point-Standard*).

```
...

"""
Basic Operations
"""
num = ...
n = ...
num &= n # AND
num |= n # OR
num ^= n # XOR
num = ~num # 'num''s complement
num <<= n # Left shift 'n' bits from 'num' (Multiply 'num' by 2^'n')
num >>= n # Right shift 'n' bits from 'num' (Divide 'num' by 2^'n'. Fills left-side bits with

"""
Tricks that help if you know them by heart
"""
num = ~num + 1 # Two's complement of 'num'
num = (num >> (n - 1)) & 1 # Get 'n'th bit
num = num | (1 << n) # Sets the 'n'th bit to 1
num = num & ~(1 << n) # Sets the 'n'th bit to 0
```

- LetCode 191. Number of 1 Bits

Math

```
import math
import random


"""
Basic Operations
"""
1 + 2 # 3
5 - 2 # 3
3 * 4 # 12
7 / 2 # 3.5
7 // 2 # 3 - quotient
7 % 2 # 1 - remainder

abs(-1) # 1
max(5, 3) # 5
min(5, 3) # 3
```

```python
"""
Additional math functions
"""
math.ceil(3.12) # 4
math.floor(3.12) # 3
math.sqrt(9) # 3
math.inf # float('Inf')
-math.inf # float(-'Inf')

"""
Random functions
"""
random.randrange(5) # Random number of range [0, 5)
random.randint(0, 3) # Random number of range [start, end]
random.random() # Random number between [0.0, 1.0)
random.shuffle([1, 3, 2, 4]) # In-place shuffles the list
random.choice([1, 3, 2, 4]) # Random single item from list
```

- LeetCode 50. Pow(x, n)
- LeetCode 204. Count Primes

List

```python
...

from copy import copy


"""
Allocation
"""
empty_list = list()
list_ = [1, 2, 3]
matrix = [
    [1, 2],
    [3, 4],
    [5, 6],
]

"""
Basic Operations
"""
list_ = [1, 6, 9, 7]
list_.append(3) # TC: Amortized O(1) - [1, 6, 9, 7, 3]
list_.insert(2, 1) # TC: O(N) - [1, 6, 1, 9, 7, 3] (If index is out-of-bounds towards lowest i
list_.remove(1) # TC: O(N) - [6, 1, 9, 7, 3] (If value does not exist, raises 'ValueError')
value_of_3rd_index = list_[2]  # TC: O(1) - 9 (index starts from 0)
```

```python
value_of_last_index = list_[-1]  # TC: O(1) - 3

len(list_) # TC: O(1) - 3
min(list_) # TC: O(N) - 9 (We mostly care about numbers, as elements in interviews)
max(list_) # TC: O(N) - 1 (We mostly care about numbers, as elements in interviews)

if 1 in list_: # TC: O(N)
    ...
for e in list_:
    ...

"""
Range

'range(..)' is an immutable sequence object (not an iterable)
It also doesn't create the materialized sequence beforehand (memory-efficient)
"""
range_from_zero_to_end = range(5) # 0, 1, 2, 3, 4
range_from_start_to_end = range(2, 5) # 2, 3, 4
range_from_start_to_end_by_skip = range(2, 5, 2) # 2, 4
list(range(100)) # [0, 1, 2, ... , 99] ('range' object is materialized only when wrapped with

"""
Copy
"""
list_ = [1, 2, 3, 4]
copied_list = copy(list_)

"""
Reverse
"""
list_ = [1, 2, 3]
reversed_ = reversed(list_) # TC: O(1) - 3, 2, 1
list_.reverse() # TC: O(N) - [3, 2, 1] (In-place)

"""
Slicing
"""
list_ = [0, 1, 2, 3, 4, 5, 6, 7]
empty_sliced = list_[2:2] # []
range_sliced = list_[2:4] # [2, 4]
inverted_sliced = list_[-3:-1] # [5, 6] (Same as slice_target_list[len(slice_target_list)-3:le
skipped_sliced = list_[1:5:2] # [1, 3] (Skip by 2 inclusive from the start index)
reverse_sliced = list_[5:1:-2] # [5, 3, 1] (Skip by -2 inclusive from the start index reverse)
rotated = list_[2:] + list_[:2] # [2, 3, 4, 5, 6, 7, 0, 1] (Rotates slice_target_list by '2' t

"""
```

```
List Comprehension and Tricks
"""
init_1d_list = [0 for _ in range(2)] # [0, 0]
init_2d_matrix = [[0 for _ in range(2)] for _ in range(2)] # [[0, 0], [0, 0]] (DO NOT USE [0]
filtered = [x for x in range(0, 3) if x % 2 == 0] # [0, 2]
flattened = [item for sublist in [[1, 2], [3, 4]] for item in sublist] # [1, 2, 3, 4]

"""
Others
"""
tuple([1, 2, 3]) # (1, 2, 3) (You can use 'tuple' as immutable list. All operations that can b
zip([1, 2, 3], ['zebra', 'tiger', 'giraffe']) # TC: O(1) - (1, 'zebra'), (2, 'tiger'), (3, 'gi
enumerate([1, 2, 3]) # TC: O(1) - (0, 1), (1, 2), (2, 3) (Useful when you need to map value ->
```

String

```
...

"""
Allocation
"""
empty_str = str()
plain_str = 'hello'
repeated_str = 'a' * 5 # "aaaaa"

"""
Basic Operations
"""
str_ = 'hello'
if 'abc' in str_: # TC: O(N)
    ...
for ch in str_:
    ...

"""
Editing

Strings are immutable, therefore it needs to be converted to list for efficient editing (and i
Note that `concat = string1 + string2` is an expensive operation as it copies both strings to
"""
char_list = list('hello') # ['h', 'e', 'l', 'l', 'o']
''.join(char_list) # 'hello'

"""
Slicing
"""
```

```
# All slicing operations in list are possible in string as well

"""
Prefix / Suffix match
"""
str_ = 'PrefixWordSuffix'
str_.startswith('Prefix') # TC: O(P) - True
str_.endswith('Suffix') # TC: O(S) - True

"""
Stringify
"""
str(1) # '1'
str(0.2) # '0.2'
str([1, 2, 3]) # '123'

"""
Numberify
"""
int('23') # 23 (int)
float('3.14') # 3.14 (float)

"""
Others
"""
tuple('hello') # ('h', 'e', 'l', 'l', 'o') (If you need to, you can use tuple as immutable lis
ord('a') # 97 (Converts character into unique integer. This is mostly useful when you want to

"""
Utility Operations for input sanitization or debug output
"""
'   hello '.strip() # 'hello'
'AbCde'.lower() # 'abcde'

str_list = 'hello,how,are,you'.split(',') # ['hello', 'how', 'are', 'you']
','.join(str_list) # 'hello,how,are,you'

name = 'Eminem'
rank = 3
f'Name: {name}, Rank: {rank}' # 'Name: Eminem, Rank: 3'
```

Set

```
...

"""
Allocation
```

```
"""
empty_set = set()
set_ = {1, 2, 3}

"""
Basic Operations
"""
set_ = {1, 5, 3, 2}
set_.add(1) # TC: O(1) - {1, 5, 3, 2}
set_.remove(1) # TC: O(1) - {5, 3, 2} (Raises 'KeyError' if value does not exist)
set_.discard(1) # TC: O(1) - {5, 3, 2} (Does not raise 'KeyError' if value does not exist)
set_converted_list = set([3, 2, 1, 3, 5]) # {3, 2, 1, 5} (Removes duplicate)

len(set_) # TC: O(1)
min(set_) # TC: O(N) (We mostly care about numbers, as element in interviews)
max(set_) # TC: O(N) (We mostly care about numbers, as element in interviews)

if 1 in set_: # TC: O(1)
    ...
for val in set_:
    ...
```

Dictionary

```
...

from collections import defaultdict, Counter


"""
Allocation
"""
empty_dict = dict()
dict_ = {'birds': 1, 'mammals': 2, 'reptiles': 1}
default_dict = defaultdict(int) # If accessed like `d[n]`, it will automatically initialize th

"""
Basic Operations
"""
dict_ = {'John': 23, 'Amy': 32, 'George': 43}
tuple_list = list(dict_) # [('John', 23), ('Amy', 32), ('George', 43)]
dict_ = dict(tuple_list) # {'John': 23, 'Amy': 32, 'George': 43} (Tuples can be converted to '
john_age = dict_['John'] # TC: O(1) - 2
dict_.pop('Amy') # TC: O(1) - 23, {'John': 23, 'George': 43} (Removes key and returns value. R
dict_.pop('Adam', 0) # TC: O(1) - 0, {'John': 23, 'George': 43} (Removes key and returns value

# Each returns its own immutable class object (it can't be indexed with 'enumerate(..)')
```

```
keys = dict_.keys()
if 'John' in dict_: # TC: O(1)
    ...
for key in dict_:
    ...
if 'John' in keys(): # TC: O(1)
    ...
for key in keys():
    ...

vals = dict_.values()
if 2 in vals: # TC: O(N)
    ...
for val in vals:
    ...

items = dict_.items()
if ('John', 23) in items: # TC: O(N)
    ...
for key, val in items:
    ...

"""
Dictionary Comprehension
"""
key_val_swapped_dict = {val: key for key, val in {1: 'a', 2: 'b', 3: 'c'}} # {'a': 1, 'b': 2,

"""
Counters

It's a convenience collection to count items. It can remove alot of boilerplate code than using
"""
Counter(['banana', 'orange', 'apple', 'apple']) # {'banana': 1, 'orange': 1, 'apple': 2}
Counter('hello') # {'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

Make sure you understand how Set / Dictionary works internally. - LeetCode 705. Design HashSet - LeetCode 706. Design HashMap

Set / Dictionary often comes as supporting data structure of a main data structure for optimization purpose (mostly for List / String in interviews). - LeetCode 1. Two Sum - LeetCode 125. Valid Palindrome

Set / Dictionary is also used to represent Graphs (covered later) and mark visited nodes.

Linked List

You will encounter in most part, 2 kinds of questions for Linked List in interviews.

- Using a predefined **singly-linked list** node class. Note that these problems are generally

solved with previous, current, next and temp pointers.

```
...

from __future__ import annotations
from typing import Any

class ListNode:

    def __init__(self, val: Any, next_: ListNode):
        self.val = val
        self.next = next_
```

  - LeetCode 206. Reverse Linked List

- Using custom **doubly-linked list** that integrates with other data structures such as Dictionary for building efficient data structures.

```
...

from __future__ import annotations
from typing import Any

class ListNode:

    def __init__(self, val: Any, prev: ListNode, next_: ListNode):
        self.val = val
        self.next = next_
        self.prev = prev
```

  - LeetCode 146. LRU Cache

Tips

- Dummy node: When using a dummy node, we don't need to check the corner case where the head is null. Definition of an empty list is now **next node of head node is null**.
- Tail node: When you track the tail node, it's easier to append new nodes. Creating a new list as a result of another operation will be a good example. If the tail node is not given with the predefined list, you will have to at least traverse the entire list once, which makes the time complexity at least O(N). Tail node can also point to a dummy node to simplify code.
- Middle node: It is often asked in interviews of finding the middle node of a singly-linked list. It is useful when we have to break the list into 2, or moving 2 pointers in different speed.

Some solutions may use all three tips - LeetCode 148. sort-list

Deque / Stack / Queue

`collections.deque` in Python is implemented as doubly-linked list (although it's never used in Linked List problems).

We will be using Deque as core data structure for both Stack / Queue because it contains all operations needed.

```
...

from collections import deque


"""
Deque
"""
deq = deque([1, 2, 3])
deq.append(4) # TC: O(1) - [1, 2, 3, 4]
deq.appendleft(0) # TC: O(1) - [0, 1, 2, 3, 4]
deq.pop() # TC: O(1) - 4, [0, 1, 2, 3]
deq.popleft() # TC: O(1) - 0, [1, 2, 3]

"""
Queue
"""
que = deque([1, 2, 3]) # Allocate deque as queue
que.append(4) # TC: O(1) - [1, 2, 3, 4] (Offer)
que.popleft() # TC: O(1) - 1, [2, 3, 4] (Poll)
peek_first = que[0] # TC: O(1) - 2

"""
Stack
"""
stk = deque([1, 2, 3]) # Allocate deque as stack
stk.append(4) # TC: O(1) - [1, 2, 3, 4] (Push)
stk.pop() # TC: O(1) - 4, [1, 2, 3] (Pop)
peek_top = stk[-1] # TC: O(1) - 3

"""
Basic Operations
"""
stk_or_que = deque()
if stk_or_que: # TC: O(1) - If stack or queue is not 'None' and `len(stack_or_queue) > 0` (Mos
    ...
while stk_or_que: # TC: O(N) - While stack or queue is not 'None' and `len(stack_or_queue) > 0
    ...
```

Deque can contain the snapshot of sliding window. - LeetCode 3. Longest Substring Without Repeating Characters

Queue is used in BFS later in Graphs.

Stack can be used for parsing structures. - LeetCode 20. Valid Parentheses

Monotonic Deque / Queue (Sliding Window / Timestamp problems) - LeetCode 239. Sliding Window Maximum - LeetCode 362. Design Hit Counter

Monotonic Stack - LeetCode 155. Min Stack - LeetCode 496. Next Greater Element I - LeetCode

503. Next Greater Element II

Heap

```
...

import heapq
from typing import List


"""
Basic Operations
"""
heap = [1, 4, 2, 5, 3]
heapq.heapify(heap) # TC: O(N) - Amortized (not O(N * logN))
heapq.heappush(heap, 0) # TC: O(logN) - [0, 1, 4, 2, 5, 3]
val = heapq.heappop(heap) # TC: O(logN) - [1, 3, 2, 5, 4]
peek = heap[0] # TC: O(1)

"""
Top / Bottom K

TC: O(N * logK)
SC: O(K)
"""
def top_k(nums: List[int], k: int) -> List[int]:
    """
    Uses min heap, so that everytime the heap is full, we pop from the smallest element
    """

    min_heap = list()
    for num in nums:
        heapq.heappush(min_heap, num)
        if len(min_heap) > k:
            heapq.heappop(min_heap)

    res = list()
    while len(min_heap) > 0:
        res.append(heapq.heappop(min_heap))
    return res


def bottom_k(nums: List[int], k: int) -> List[int]:
    """
    Uses max heap, so that everytime the heap is full, we pop from the largest element
    """
```

```
    max_heap = list()
    for num in nums:
        heapq.heappush(max_heap, -num)
        if len(max_heap) > k:
            heapq.heappop(max_heap)

    res = list()
    while len(max_heap) > 0:
        res.append(-heapq.heappop(max_heap))
    return res
```

Keep in mind that there is no point using Heap if you just throw all elements in it to find the top-K / bottom-K elements since it has no benefit over sort-and-iterate in terms of time / space complexity. Key is to keep the Heap size as 'K' by removing the next value after inserting a new element. - LeetCode 215. Kth Largest Element in an Array

Oftentimes, Heaps are used for quick access, update of min / max values while iterating a list. - LeetCode 253. Meeting Rooms II - LeetCode 1438. Longest Continuous Subarray with Absolute Diff Less Than or Equal to Limit

Divide and Conquer

Problems that have *Optimal Substructure* (problems where solution to the original problem can be constructed from the solutions of its sub-problems) can be solved using Divide and Conquer. Problems that are solved with Divide and Conquer *do not* have *Overlapping Sub-problems*, which we will discuss later in Dynamic Programming chapter.

Sort / Binary Search are great examples of Divide and Conquer.

Sort

There are several sorting algorithms with quadratic runtime (*Bubble Sort*, *Insertion Sort*, *Selection Sort*). However, due to its penalty in time complexity, improved algorithms like below are more likely to be in interviews.

- *Merge Sort*

```
...

from typing import List


def merge_sort(nums: List[int]):
    """
    Stable sorting algorithm (elements with same key preserves ordering) using Divide and
    The divide step is implemented in simple recursion top-down, while the merge step merg

    TC: O(N*logN)
    SC: O(N)
```

```
    """

    def merge(lo: List[int], hi: List[int]):
        i, j, k = 0, 0, 0

        while i < len(lo) and j < len(hi):
            if lo[i] <= hi[j]:
                nums[k] = lo[i]
                i += 1
            else:
                nums[k] = hi[j]
                j += 1
            k += 1

        # Drain rest of 'lo'
        while i < len(lo):
            nums[k] = lo[i]
            i += 1
            k += 1

        # Drain rest of 'hi'
        while j < len(hi):
            nums[k] = hi[j]
            j += 1
            k += 1

    if len(nums) <= 1:
        return

    mid = len(nums) // 2
    left = nums[:mid]
    right = nums[mid:]

    merge_sort(left)
    merge_sort(right)
    merge(left, right)
```

The more generic version of merge step will be *K Way Merge* where it merges K Lists.

  - LeetCode 23. Merge k Sorted Lists

- *Quick Sort*

```
...

  from typing import List


def quick_sort(nums: List[int]):
```

```python
    """
    Unstable sorting algorithm (elements with same key does NOT preserve ordering) using l
    Most widely adapted sorting algorithm due to its performance

    TC: O(N*logN)
    SC: O(logN)
    """

    def sort(lo: int, hi: int):

        def hoare_partition(lo: int, hi: int) -> int:
            i, j = lo - 1, hi + 1
            piv = nums[lo]
            while True:
                i += 1
                while nums[i] < piv:
                    i += 1
                j -= 1
                while nums[j] > piv:
                    j -= 1
                if i >= j:
                    break
                nums[i], nums[j] = nums[j], nums[i]
            return j

        def lomuto_partition(lo: int, hi: int) -> int:
            piv = nums[hi]
            i = lo
            for j in range(lo, hi):
                if nums[j] <= piv:
                    nums[i], nums[j] = nums[j], nums[i]
                    i += 1
            nums[i], nums[hi] = nums[hi], nums[i]
            return i

        if lo >= hi:
            return

        # pivot_idx = hoare_partition(lo, hi)
        pivot_idx = lomuto_partition(lo, hi)
        sort(lo, pivot_idx - 1)
        sort(pivot_idx + 1, hi)

    sort(0, len(nums) - 1)
```

- *Counting Sort*
- *Radix Sort*

Revisiting 'finding top-K / bottom-K elements' using *Quick Select* (partitioning part of *Quick Sort*) or *Counting* (counting part of *Counting Sort*) to achieve near O(N) performance. - LeetCode 215. Kth Largest Element in an Array

```
"""
Basic Operations

Python uses Tim Sort
TC: O(NlogN)
SC: O(N) Average
"""
nums = [3, 1, 6, 5]
nums.sort() # [1, 3, 5, 6] (In-place)
sorted_nums = sorted(nums) # [1, 3, 5, 6] (New list, not an iterator)

sorted_word = sorted('bacd') # ['a', 'b', 'c', 'd']

tuple_list = [(1, 3), (3, 2), (2, 1)]
sorted_by_first = sorted(tuple_list, key=lambda x: x[0]) # [(1, 3), (2, 1), (3, 2)]
sorted_by_second = sorted(tuple_list, key=lambda x: x[1]) # [(2, 1), (3, 2), (1, 3)]
tuple_list.sort() # [(1, 3), (2, 1), (3, 2)]
tuple_list.sort(key=lambda x: x[0]) # [(1, 3), (2, 1), (3, 2)]
tuple_list.sort(key=lambda x: x[1]) # [(2, 1), (3, 2), (1, 3)]
```

Binary Search

```
import bisect


"""
Basic Operations

TC: O(logN)
"""
nums = [0, 1, 2, 2, 3, 6]
left = bisect.bisect_left(nums, 2) # TC: O(logN) - 2 (Returns the left-most place in the sorte
right = bisect.bisect_right(nums, 2) # TC: O(logN) - 5 (Returns the right-most place in the so
ranged_left = bisect.bisect_left(nums, 12, lo=1, hi=5) # TC: O(logN) - 5 (Parameters allow ran
ranged_right = bisect.bisect_right(nums, 12, lo=1, hi=5) # TC: O(logN) - 5 (Parameters allow r

if left == right: # It means value is not found
    ...
if left != right: # It means value is found
    # To find the actual index of the first and last occurrence of the 'val', it should be eit
    ...
```

- Almost all Binary Search questions involve using either
  - Lists that is given either sorted or unsorted.

* Not sorted: Need to use the built-in sort function and then think about what to apply with the sorted List.
  - LeetCode 56. Merge Intervals
  - LeetCode 242. Valid Anagram
* Sorted: The intention of these questions are to reach the optimal time complexity (O(logN)) with a slightly tweaked sorted structure. This means it's to either use the built-in Binary Search method or implement a customized version yourself to meet the use case.

```
...

from typing import List

"""
This function is simplified (but customizable) Binary Search function.
There are problems that require you to tweak this function in order to solve the p
"""
def binary_search(nums: List[int], target) -> int:
    lo = 0
    hi = len(nums) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2 # Prevent overflow

        # Target is found (tweak is often required here)
        if nums[mid] == target:
            return mid

        # Tune 'lo' and 'hi' if pivot is not found
        if nums[mid] <= nums[hi]:
            hi = mid - 1
        elif nums[mid] >= nums[lo]:
            lo = mid + 1
    return -1
```

  - LeetCode 33. Search in Rotated Sorted Array
  - LeetCode 34. Find First and Last Position of Element in Sorted Array
* Root node of a Binary Tree that is sorted when traversed in-order.

Self-Balancing Binary Search Tree

So far, we've looked into Sort / Binary Search on non-changing data sets since elements need to be only sorted once for Binary Search(es). However, when elements are dynamically added / deleted, sorting the entire List every time is not efficient. To address this issue, we use a special data structure called Self-Balancing Binary Search Tree. Self-Balancing Binary Search Tree is a unique data structure not only because it keeps the ordering of elements, but it also self balances when you add / delete elements. This is an important feature since without it, the tree will eventually get imbalanced and the search can take up to O(N) in worst case (same as linear scan). Below is the time complexity of core operations on *Red Black Tree* (a variant of Self-Balancing Binary Search Tree). - Insert O(logN) - Delete O(logN) - Access O(logN)

image

Figure 1: image

image

Figure 2: image

Python does not have a built-in Red-Black tree. However, there is a widely used 3rd-party library called sortedcontainers and is generally allowed in interviews. - LeetCode 1244. Design A Leaderboard - LeetCode 729. My Calendar I - LeetCode 731. My Calendar II

Binary Tree

- *Root Node*: Node that is on the first level of the Binary Tree.
- *Leaf Node*: Nodes that do not have any children.
- *Depth* and *Height*: Height starts from Leaf Node, Depth starts from Root Node, thus **height of a tree == maximum depth of subtree**.
  - LeetCode 110. Balanced Binary Tree
- Various strains of Binary Trees.

  Heap and Binary Search Tree are Complete Binary Tree. Perfect Binary Tree contains `2^(h + 1) + 1` nodes and `2^h` Leaf Nodes.

Traversal

Almost all Binary Tree problems are given a Root Node that represents the tree, and require various traversals.

```
...

from __future__ import annotations
from typing import Any, List, Optional
from collections import deque


"""
N = number of Nodes in tree

TC: O(N)
SC: O(logN)
"""

class TreeNode:

    def __init__(self, val: Any, left: Optional[TreeNode], right: Optional[TreeNode]):
        self.val = val
        self.left = left
```

```python
        self.right = right


def inorder_recursive(root: TreeNode) -> List[Any]:
    res = list()

    def recurse(_root: TreeNode):
        if root is None:
            return

        recurse(_root.left)
        res.append(_root)
        recurse(_root.right)

    recurse(root)
    return res


def inorder_iterative(root: TreeNode) -> List[Any]:
    res = list()
    stk = deque()
    cur = root
    while stk or cur:
        while cur:
            stk.append(cur)
            cur = cur.left
        cur = stk.pop()
        res.append(cur)
        cur = cur.right
    return res


def preorder_recursive(root: TreeNode) -> List[Any]:
    res = list()

    def recurse(_root: TreeNode):
        if _root is None:
            return

        res.append(_root)
        recurse(_root.left)
        recurse(_root.right)

    recurse(root)
    return res
```

```python
def preorder_iterative(root: TreeNode) -> List[Any]:
    if root is None:
        return list()

    res = list()
    stk = deque()
    stk.append(root)
    while stk:
        cur = stk.pop()
        res.append(cur.val)

        # Push right first so that left is processed first
        if cur.right:
            stk.append(cur.right)
        if cur.left:
            stk.append(cur.left)
    return res


def postorder_recursive(root: TreeNode) -> List[Any]:
    res = list()

    def recurse(_root: TreeNode):
        if root is None:
            return

        recurse(_root.left)
        recurse(_root.right)
        res.append(_root)

    recurse(root)
    return res


def postorder_iterative(root: TreeNode) -> List[Any]:
    if root is None:
        return list()

    res = list()
    stk = deque()
    cur = root
    last = None
    while stk or cur:
        if cur:
            stk.append(cur)
            cur = cur.left
        else:
```

```
            peeked = stk[-1]
            if peeked.right and last != peeked.right:
                cur = peeked.right
            else:
                res.append(peeked)
                last = stk.pop()
    return res


def level_order(root: TreeNode) -> List[Any]:
    res = list()
    que = deque()
    que.append(root)
    while que:
        size = len(que)
        while size > 0:
            polled = que.popleft()
            res.append(polled)
            if polled.left:
                que.append(polled.left)
            if polled.right:
                que.append(polled.right)
            size -= 1
    return res
```

Problems often require you to construct / reconstruct a Binary Tree. - LeetCode 108. Convert Sorted Array to Binary Search Tree - LeetCode 226. Invert Binary Tree

In-order with either Pre-order or Post-order traversal, a unique Binary Tree can be constructed from a sorted List. - LeetCode 105. Construct Binary Tree from Preorder and Inorder Traversal

However, In-order alone or Pre-order with Post-order traversal cannot construct a unique Binary Tree. - LeetCode 889. Construct Binary Tree from Preorder and Postorder traversal

With the help of 'null' nodes marked as special characters, serialized representation of a Binary Tree can be derived from Pre-order or Post-order traversal. - LeetCode 572. Subtree of Another Tree - LeetCode 652. Find Duplicate Subtrees

There is an In-order traversal algorithm called Morris Inorder Traversal that uses O(1) space by not using recursion or stack but iteration.

Graph

- *Vertex*: Nodes in the Graph.
- *Connected*: Graph is considered **Connected** if every pair of vertices in the Graph is directly or indirectly reachable.
- *Edge*: Relation between vertices. Edges are always `vertices - 1` if the Graph is connected. Edges can be weighted (with both positive and negative values) and these Graphs are called **weighted** Graph. However, most problems in interviews will deal with **non-weighted** Graph.

topological_ordering

Figure 3: topological_ordering

Categories & Must-Knows

- *Undirected Graph*: Graph that contains vertices with no direction.
  - *Disjoint Set (Union Find)*: Data structure that group vertices that are connected in Undirected Graph.

```
...

from typing import Any, Dict, Set


class DisjointSet:

    def __init__(self, graph: Dict[Any, Set[Any]]):
        self.vtx_to_parent = {vtx: vtx for vtx in graph.keys()}
        self.vtx_to_rank = {vtx: 0 for vtx in graph.keys()}
        for vtx, nbrs in graph.items():
            for nbr in nbrs:
                self.union(vtx, nbr)

    def union(self, vtx1: Any, vtx2: Any):
        parent1 = self.find(vtx1)
        parent2 = self.find(vtx2)
        if parent1 != parent2:
            if self.vtx_to_rank[parent1] > self.vtx_to_rank[parent2]:
                self.vtx_to_parent[parent2] = parent1
            elif self.vtx_to_rank[parent1] < self.vtx_to_rank[parent2]:
                self.vtx_to_parent[parent1] = parent2
            else:
                self.vtx_to_parent[parent2] = parent1
                self.vtx_to_rank[parent1] += 1

    def find(self, vtx: Any) -> Any:
        parent = self.vtx_to_parent[vtx]
        if parent != vtx:
            parent = self.find(parent)
            self.vtx_to_parent[vtx] = parent
        return parent
```

    Also used for cycle detection in Undirected Graph.
    * LeetCode 684. Redundant Connection
    * LeetCode 685. Redundant Connection II
- *Directed Graph*: Graph that contains vertices with direction.
  - *Topological Sort*: Ordering of the vertices where each edge is from the *vertex earlier in the ordering* to *vertex later in the ordering*. It represents the dependency Graph of vertices (thus can detect cycles in Directed Graphs).

trie

Figure 4: trie

```
...

from typing import Any, Dict, Set, List, Deque
from collections import defaultdict, deque


def topsort(graph: Dict[Any, Set[Any]]) -> List[Any]:
    """
    Using Khan's Algorithm
    """

    # Build-up all in-degree (edge that connects toward the vertex) counts of each ve
    in_degree = defaultdict(int)
    for vtx in graph:
        for nbr in graph[vtx]:
            in_degree[nbr] += 1

    # Initialize queue with vertices that have in-degree of 0 (starting vertices)
    que = deque([vtx for vtx in graph if in_degree[vtx] == 0])

    # Remove in-degree one-by-one and add neighbouring vertices only if there is no i
    top_sorted = list()
    while que:
        start = que.popleft()
        top_sorted.append(start)
        for nbr in graph[start]:
            in_degree[nbr] -= 1
            if in_degree[nbr] == 0:
                que.append(nbr)

    # If cycle exist, queue will be empty earlier and terminate before adding everyth
    # For example, for graph 'A -> B -> A', initialized queue will be empty as there
    if len(graph) > len(top_sorted):
        raise Exception('Cycle detected in graph')

    return top_sorted
```

  * LeetCode 207. Course Schedule
  * LeetCode 269. Alien Dictionary

- *Trie*: Tree that is optimized for matching prefix of words.

```
...

class Trie:
```

adjmatrix

Figure 5: adjmatrix

```python
class _Node:

    def __init__(self):
        self.children = dict()
        self.end = False

def __init__(self):
    self._root = self._Node()

def insert(self, word: str):
    cur = self._root
    for ch in word:
        if ch not in cur.children:
            cur.children[ch] = self._Node()
        cur = cur.children[ch]
    cur.end = True

def start_with(self, prefix: str) -> bool:
    cur = self._root
    for ch in prefix:
        if ch not in cur.children:
            return False
        cur = cur.children[ch]
    return True

def longest_prefix(self) -> str:
    prefix = list()
    cur = self._root
    while len(cur.children) == 1:
        for ch, _ in cur.children:
            if ch not cur.end:
                prefix.append(ch)
                cur = cur.children[ch]
            else:
                return ''.join(prefix)
    return ''.join(prefix)
```

* LeetCode 208. Implement Trie
* LeetCode 14. Longest Common Prefix

Representation

- *Adjacency Matrix*: Represents the Graph in N x N matrix.
  - LeetCode 1615. Maximal Network Rank

- *Adjacency Set*: Represents mapping of vertex -> set of neighboring vertices.

```
...

graph = {
  'a': {'b', 'c', 'd'},
  'b': {'e', 'f'},
}
```

Sometimes, only edges are given so we need to build the Graph from scratch. You can build the Graph in either way.

Search

- *Depth First Search (DFS)*: Recursion is used for implementation. In fact, DFS is a generalization of Pre / Post / In-order traversals of Binary Tree in previous chapter (as Graph is also one form of generalization of a N-ary Tree)

```
from typing import Any, Dict, Set, List
from collections import deque


def dfs(graph: Dict[Any, Set[Any]], start_vtx: Any) -> List[Any]:
    """
    TC: O(|V| + |E|)
    SC: O(|V|) (Adjacency List) / O(|V| ^ 2) (Adjacency Matrix)
    """

    res = list()
    seen = set()

    def recurse(vtx: Any):
        if vtx in seen:
            return

        seen.add(vtx)
        res.append(vtx)
        for nbr in graph[vtx]:
            recurse(nbr)

    def iterate(vtx: Any):
        stk = deque()
        seen.add(start_vtx)
        stk.append(start_vtx)
        while stk:
            cur = stk.pop()
            res.append(cur)
            for nbr in graph[cur]:
                if nbr in seen:
                    continue
```

```
                seen.add(nbr)
                stk.append(nbr)

    recurse(start_vtx)
    #iterate(start_vtx)
    return res
```

  - LeetCode 200. Number of Islands

- *Breadth First Search (BFS)*: Queue is used for implementation. Used for find the shortest path between 2 vertices, minimal steps to reach a target etc. In fact, BFS is a generalization of Level-order traversal of Binary Tree in previous chapter (as Graph is also one form of generalization of a N-ary Tree)

```
...

from typing import Any, Dict, Set, List
from collections import deque


def bfs(graph: Dict[Any, Set[Any]], start_vtx: Any) -> List[Any]:
    """
    TC: O(|V| + |E|)
    SC: O(|V|) (Adjacency List) / O(|V| ^ 2) (Adjacency Matrix)
    """

    res = list()
    seen = set()

    que = deque()
    seen.add(start_vtx)
    que.append(start_vtx)
    while que:
        cur = que.popleft()
        res.append(cur)
        for nbr in graph[cur]:
            if nbr in seen:
                continue

            seen.add(nbr)
            que.append(nbr)
    return res
```

  - LeetCode 934. Shortest Bridge
  - LeetCode 322. Coin Change

Advanced Graph Topics

These are advanced Graph topics. Please learn only if you have enough time until interviews.

Djikstra & *Bellmanford* - LeetCode 787. Cheapest Flights Within K Stops

*A** - LeetCode 1091. Shortest Path in Binary Matrix

Minimum Spanning Trees: Tree where the sum of all edge weights are minimum by picking the edges in Greedy manner. In general, the algorithm is designed toward weighted Graph. However for non-weighted Graph, it just means the minimum count of edges. *Kruskal's Minimum Spanning Tree* (For undirected graph) & *Prim's Minimum Spanning Tree* (For directed graph) are examples. - LeetCode 1584. Min Cost to Connect All Points

Backtracking

Backtracking is a technique for solving problems where you explore all possible options and backtrack as soon as you determine the current path cannot lead to a valid solution (pruning). For this reason, DFS becomes handy for these problem solution, thus often resulting in exponential time complexity.

Difference between Recurrence Relation

Backtracking and Recurrence Relation (e.g Fibonacci, House Robber without Dynamic Programming) problems are often confused to be in same category, as both often involve recursion. However, problems that require Backtracking creates a decision tree for a large solution space (e.g get all subsets [large solution space] from a list of numbers by either selecting or not selecting each element in list [decision tree]). This trait often leads Backtracking solutions to undo decisions in order to explore alternate paths. - LeetCode 78. Subsets - LeetCode 90. Subsets II - LeetCode 46. Permutations - LeetCode 47. Permutations II - LeetCode 77. Combinations - LeetCode 39. Combination Sum - LeetCode 40. Combination Sum II - LeetCode 79. Word Search

Tips

When the problem asks for a result where all exploration paths matter (computing all combination, route of a graph etc), it is highly likely be solved with Backtracking.

Make sure you draw the call graph for better understanding for yourself as well as the interviewer.

Dynamic Programming

Simply put, Dynamic Programming is an optimization technique for *Optimal Substructure* problems where there exist *Overlapping Sub-problems*. Any problem (mostly likely will be Recurrence Relation or Backtracking problems in interviews) that satisfies this condition can be a candidate for optimization with Dynamic Programming. Divide and Conquer solutions discussed in previous chapter (merge sort, binary search etc) by definition has no Overlapping Sub-problems, thus is not a candidate for optimization.

Let's revisit Fibonacci Sequence calculation on how Dynamic Programming can optimize this problem.

- *DFS without cache*

```
...

def fib(num: int) -> int:
```

```
    """
    TC: O(2^N)
    SC: O(N)

    Constraint: 0 <= num <= 45
    """

    if num <= 1:
        return num

    return fib(num - 1) + fib(num - 2)
```

- *Memoization* (DFS with cache)

```
...

def fib_memoized(num: int) -> int:
    """
    Starts from the original problem and descends toward the base-case (hence top-down app

    TC: O(N)
    SC: O(N)

    Constraint: 0 <= num <= 45
    """

    memo = [-1 for _ in range(num + 1)]

    def dfs(n: int) -> int:
        if n == 0:
            return 0
        if n == 1:
            return 1

        if memo[n] != -1:
            return memo[n]

        memo[n] = dfs(n - 1) + dfs(n - 2)
        return memo[n]

    return dfs(num)
```

- *Tabulation*

```
...

def fib_tabulation(num: int) -> int:
    """
    Starts from the base-case and ascends toward the original problem (hence bottom-up app
```

```
    TC: O(N)
    SC: O(N)

    Constraint: 0 <= num <= 45
    """

    if n <= 1:
        return n

    dp = [-1 for _ in range(num + 1)]

    dp[0] = 0
    dp[1] = 1
    for i in range(2, num + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[num]


def fib_tabulation_space_optimized(num: int) -> int:
    """
    TC: O(N)
    SC: O(1)

    Constraint: 0 <= num <= 45
    """

    if n <= 1:
        return n

    prv = 0
    cur = 1
    for _ in range(2, num + 1):
        prv, cur = cur, cur + prv # This line is not sequential. Left-side 'prv' and righ
    return cur
```

- Recurrence Relation problems: Recurrence Relation problems are classic problems that can be optimally solved with Dynamic Programming.
  - LeetCode 70. Climbing Stairs
  - LeetCode 198. Min Cost Climbing Stairs
  - LeetCode 198. House Robber
  - LeetCode 213. House Robber II
  - LeetCode 1143. Longest Common Subsequence
- Sub-list / Sub-string problems: Tabulation is mostly used to solve these problems.
  - LeetCode 53. Maximum Subarray
  - LeetCode 121. Best Time to Buy and Sell Stock
  - LeetCode 238. Product of Array Except Self

- Backtracking problems: Backtracking problems can be solved optimally with Dynamic Programming however, if constraints get complicated, it's not easy to identify overlapping sub-problems. In most cases, Backtracking solution will be enough to pass the interview however, there are some Backtracking problems where overlapping sub-problems are easier to identify, thus can be further optimized with Dynamic Programming.
  - LeetCode 55. Jump Game
  - LeetCode 45. Jump Game II

Tips

- Memoization should come up first as solution because it's more intuitive (non-memoized version is basically the plain DFS function). However, as Tabulation often is more efficient in terms of space complexity (since it does not use the recursion call stack), try solving the problem first with Memoization, and then optimize towards Tabulation if interviewer explicitly asks for a bottom-up approach. Tabulation, as explained above can also (but not always) lead to further space optimization as well. If you have defined the base case well, both Memoization and Tabulation will share the same base case and similar code structure.
- If the problem asks for anything in context of min / max / number-of-XYZ etc that requires exploration OR anything in context of prefix sum, it is highly likely to be solved optimally with Dynamic Programming. As described in the first bullet-point, when initially coming up with a DFS function to solve these problems, the function will often return a type of either `int` or `bool`. These return values are likely cachable, thus eligible for Memoization.

Greedy

Greedy is an algorithm pattern that chooses local optimal choice on each stage and never change its decision in the following stages. You can use Greedy to find an optimal solution if the problem has *Optimal Substructure* (discussed in Divide and Conquer) and *Greedy Choice Property* (an overall optimal solution can be reached by choosing the optimal choice at each step).

To name a few difference from Dynamic Programming - Dynamic Programming and Greedy can be both used for optimizing solution for problems having *Optimal Substructure*. - Dynamic Programming optimize the solution by removing *Overlapping Problems*, however Greedy optimize the solution by applying the Greedy Choice Property. - Dynamic Programming usually can solve problems that can be solved by Greedy, however is preferable to solve with Greedy if Greedy Choice Property can be met, as it's almost always more optimal than Dynamic Programming.

Therefore on optimization, always try Greedy first and if it doesn't seem to work out, quickly switch to Dynamic Programming. Unfortunately there is no straight-forward way to tell if a problem meets the Greedy Choice Property (you just need to try it).

Greedy often use the Heap, as it offers efficient min / max element on each stage. - LeetCode 1167. Minimum Cost to Connect Sticks

You may occasionally see a Greedy solution that optimizes Dynamic Programming solution even further. - LeetCode 55. Jump Game - LeetCode 45. Jump Game II

Tips

If the problem looks like it can be solved with Dynamic Programming, quickly try a Greedy solution if it works. If it doesn't, fallback to Dynamic Programming.

Two Pointers

Two Pointers technique is an interesting technique that often are in used in combination with other concepts and techniques to reach an optimal solution.

Linked List - LeetCode 876. Middle of the Linked List

Binary Search - LeetCode 167. Two Sum II - Input Array is Sorted

Dictionary - LeetCode 125. Valid Palindrome

Sliding Window - LeetCode 567. Permutation in String

Complexity Theory

Dynamic Programming and Greedy patterns help us achieve *Polynomial Runtime* on problems that otherwise require *Exponential Runtime*. However, there are vast amount of problems that the only-known current solution has an Exponential Runtime. This is the part where Complexity Theory comes in to classify problems based on their status of time complexity. Make sure to understand the concepts of *P / NP*, *NP Complete* and *NP Hard*. NP Complete or NP Hard problems are out of scope for interviews since proving that the problem is NP Complete or NP Hard is non-trivial. However, if you know some well-known algorithms, you can at least mention to the interviewer that the problem cannot be solved with Polynomial Runtime. Here are some well-known problems.

0/1 Knapsack Problem (*NP Complete*): Can be solved both ways, Top-down / Bottom-up - LeetCode 416. Partition Equal Subset Sum

Travelling Salesmen Problem (*NP Complete* for decision / *NP Hard* for optimization) - LeetCode 943. Find the Shortest Superstring

Concurrency

Concurrent programming offers seamless experience as if multiple programs are running at the same time (even with single core CPU). However, the most interesting part in this subject is blocking operations, along with the problems that follow in order to prevent it.

When processes or threads go into block state, the operating system scheduler reschedules them to run other processes or threads. Yet, rescheduling of processes or threads is expensive due to context changes, and too many blocked threads (often happens in IO-heavy operations) can quickly fill up the task queue, which can lead to OOM. To resolve this, there were many asynchronous paradigms that emerged in the past decade (with its own problems). To name a few

- *Callbacks*: When thread blocks, thread is returned back to thread pool and reassigned on call-back functions. This however led to problems like call-back hell.
- *Reactive (Rx)*: Using the Observer Pattern, Reactive paradigms doesn't suffer from call-backs, however had its own problem of composing streams and the huge learning curve behind it.

- *Coroutines*: Coroutines paradigm take a different approach where the language provides its own building blocks of the program with something called Coroutines. Each Coroutine call will suspend the caller Coroutine until it finishes, but never blocks the thread underneath so that the thread can immediately start executing other Coroutines. This is similar in the way Processes or Threads are rescheduled by the operating system scheduler, however Coroutines do not affect the operating system scheduler because from operating system's point-of-view, the underlying thread is always active (thus Coroutines switches are cheap). Coroutines are also easier to debug than Callbacks and Reactive approaches.

```
...

import asyncio
import time


"""
We will be focusing on Coroutines, as it's the de-facto way of asynchronous programming today
"""

# This is a single Coroutine block
async def get_data_by_id(uuid: int) -> str:
    time.sleep(1)
    return f'{uuid}: data'


# This is another Coroutine block that calls the other Coroutine
async def get_data_with_uuid_1() -> str:
    return await get_data_by_id(1) # Once invoked with 'await', other Coroutines are given cha


loop = asyncio.get_event_loop()
loop.run_until_complete(get_data_with_uuid_1())
```

Parallelism

Parallel programming can greatly enhance performance of a program if tasks have minimal dependencies and overlapping tasks. However, the most interesting part in this subject is race conditions, along with the problems that follow in order to prevent it.

When using locks, it's easy to write code that contain Deadlocks. Here are some generic remedies.

- *Prevent* it from happening by using timeouts when acquiring locks.
- *Detect-and-cure* by maintaining a dependency graph for threads and periodically check cycles.

However, careful use of locks must be practiced before relying on generic remedies.

There are other paradigms to avoid race conditions, however it's only worth just mentioning it to the interviewer to demonstrate your knowledge of options.

- Actors

- Purely Functional Programming

```
...

from threading import Thread, Semaphore


"""
Threads

We assume free-thread Python (GIL turned off)
"""
thread = Thread()

"""
Semaphores
"""
# Semaphores have internal counter where if it goes down to a negative value, it will block (n
sem = Semaphore(5) # Up to 5 threads can acquire this Semaphore.
with sem:
    ...
```

There are other convenient locks such as `Lock()`, `Rlock()` or `Condition()` that Python provides, however you can do the same (or similar) by composing multiple Semaphores. Please refer to open-sourced material, *Little Book of Semaphores* for details on how Semaphores are composed to solve race conditions and deadlocks for various problems. *Chapter 4. Classical Synchronization Problems* will be enough to understand the basics.

Database Isolation Levels

As demonstrated above in Python code, race conditions that need to be resolved in-memory applies the same way in database systems. In particular, databases categorized these traits in terms of isolation levels and it is up to the programmer, which to use for the application based on the pros and cons.

Before going into each isolation level, here is a small glossary.

- *Query*: Single database operation
- *Transaction*: Entire transaction that may be composed of several *Queries*

Read Committed (Using Locks)

- No dirty read / writes (all other committed transactions are shown during current transaction).
  - Readers and writer block each other by using single lock to a row during the *query*.
  - Phantoms may show in read-only queries during transaction (read-skew).
  - Phantoms may show in series of read-modify-write queries during transaction (write-skew).
- Lost updates can happen on read-modify-and-write (race condition).
  - Need to use either lock (Pessimistic Concurrency Control) or compare-and-set (Optimistic Concurrency Control) on the row(s) in the write *query* (and error-prone) during the transaction. When using compare-and-set, high contention towards the same row can lead to a lot of transaction restarts.

Read Committed (Using MVCC)

- No dirty read / writes (all other committed transactions are shown during current transaction).
  - Readers do not block writer and writer does not block readers. Implementation may differ based on using (Pessimistic Concurrency Control) or compare-and-set (Optimistic Concurrency Control).
  - Phantoms may show in read-only queries during transaction (read-skew).
  - Phantoms may show in series of read-modify-write queries during transaction (write-skew).
- Lost updates can happen on read-modify-and-write (race condition).
  - Need to use either lock (Pessimistic Concurrency Control) or compare-and-set (Optimistic Concurrency Control) on the row(s) in the write *query* (and error-prone) during the transaction. When using compare-and-set, high contention towards the same row can lead to a lot of transaction restarts.

Snapshot Isolation (Using MVCC)

- No dirty read / writes (only rows of committed transactions before the current transaction are shown).
  - Readers do not block writer and writer does not block readers. Implementation may differ based on using (Pessimistic Concurrency Control) or compare-and-set (Optimistic Concurrency Control).
  - Phantoms *do not* show in read-only queries during transaction (read-skew).
  - Phantoms may show in some read-modify-write queries during transaction (write-skew).
- Lost updates can happen on read-modify-and-write (race condition).
  - Need to use either lock (Pessimistic Concurrency Control) or compare-and-set (Optimistic Concurrency Control) on the row(s) in the write *query* (and error-prone) during the transaction. When using compare-and-set, high contention towards the same row can lead to a lot of transaction restarts.

Serialized (Pessimistic Concurrency Control in transaction level)

- No dirty read / writes (other transactions *cannot* happen in parallel), lost updates, phantoms.
  - Readers and writer both block each other by using single lock to a row during the *transaction* (Notice the different from Read Committed, that it's the entire *transaction*). Most widely known locks are 'Two-Phase Lock (2PL)', 'Predicate Lock' and 'Index-Range Lock'. They all suffer from performance penalty due to locks.

Serializable Snapshot Isolation (Optimistic Concurrency Control in transaction level)

- No dirty read / writes (other transactions *can* happen in parallel), lost updates, phantoms.
  - Implements special algorithm that detects all other affected transactions when write is committed during the transaction (and potentially restart them).
  - High contention towards the same row can lead to a lot of transaction restarts.

Remarks on Concurrency and Parallelism

It's best to combine both asynchronous and parallel programming for optimizations. For example, - If your application is mostly waiting on IO (disk access, network etc), concurrency will be enough however, you'd want to minimize thread waiting time through asynchronous programming. - If

your application is composed of tasks are not overlapping in general, parallel programming can be a boon.

Modern web applications are a good example of mix-of-both, where each request is served in parallel however, each thread does not block on IO.

Coding Tips

- Below are frequently used variable name pools.

```
...

seen # Replacement of 'visited' (set)
deq, que, stk # 'deque' / 'queue' / 'stack'
num, nums # 'number(s)'
str_, strs # 'string(s)'
ch, chs # 'character(s)'
nbr, nbrs # 'neighbour(s)'. Used in Graphs
src, dst # 'source' / 'destination'. Used in Graphs
i, j, k # Generic indexes
lo, mid, piv, hi # 'low' / 'middle' / 'high / 'pivot' (3 pointers but handling lower / mid
cur, prv, nxt # 'current' / 'previous' / 'next' (3 pointers advancing)
row, col # 'row' / 'column'
memo, dp # 'memoization'(top-down) 'dynamic programming' (bottom-up)
val # 'value'
lvl # 'level
amt # 'amount'
targ # 'target'
cnt # 'count'
res # 'result'
rem # 'remaining'
```

- Always use single quote (') than double quotes (") as it's repetitive to press the shift key everytime for double quotes.

```
...

'string' # (O)
"string" # (X)
```

- If two or more words are unavoidable (like adjectives or 2 variables with same type), use snake case to concatenate them.

```
...

awesome_num = 3
```

- Avoid keywords for single-word variables. If you need to use them, please append '_'. Of course this doesn't apply for 2-or-more-words variables.

```
...

str_ = str()
```

```
max_ = max([1, 2, 3])
min_ = min([1, 2, 3])
sum_ = sum([1, 2, 3])
len_ = len([1, 2, 3])
```

- Some words are easily readable when read-out-loud compressed. If so, use it. However, some aren't. In this case, just use the entire word.

```
...

comp = 2 # 'complement' (easily readable)
carry = 3 # 'crry' (not easily readable. Even just removing all vowels from 'carry' doesn
```

- For Dictionaries, try using 'a_to_b' (except for counters).

```
...

from collections import Counter

val_to_i = { 'val': 1 }
str_counter = Counter('abcd')
chs_counter = Counter(['a', 'b', 'c', 'd'])
```

- Rather than Enums, use constants to save time

```
...

from enum import Enum

interval = range(10)

"""
Not recommended as it needs alot more coding
"""
class Interval(Enum):
    START, END = 0, 1
interval[Interval.START.value]
interval[Interval.END.value]

"""
Recommended as it's simpler
"""
START, END = 0, 1
interval[START]
interval[END]
```

- When nested function has clashing name as the outer function, prepend '_'. Of course if it make sense to compress it, please do so as the '_' is not needed then.

```
...

def func1(num: int):
```

```
    def func2(_num: int):
        ...
```

  – Use single-line swap, as it removes the need to introduce another variable.

```
...

a, b = b, a
```

- In interviews, there are rarely cases where you will need to define a custom class (with operand overrides) over primary type for the encapsulation of data group. However just in case you need to, here is an example of a 'Contact' class.

```
...

class Contact:

    def __init__(self, name: str, age: int):
        self.name = name
        self.age = age

    def __hash__(self) -> int:
        # Hash function that is used for data structures like 'set' / 'dict' to determine
        return hash((self.name, self.age))

    def __eq__(self, other: 'Contact') -> bool: # When self-referencing class within class
        # Used when '==' or 'sort()'
        return self.name == other.name and self.age == other.age

    def __lt__(self, other: 'Contact') -> bool:
        # Used when '<' or '>' or 'sort()'
        return (self.name, self.age) < (other.name, other.age)

    def __repr__(self) -> str:
        return f'Contact(name={self.name}, age={self.age})'
```